

Aula 3 — ASP.NET Core + MVC

Carga horária: 4 horas

Projeto base: Aula01.Web

Objetivo da aula:

Compreender o funcionamento interno do **ASP.NET Core MVC**, dominar o **pipeline HTTP**, **middleware**, **controllers**, **model binding**, **validação** e **filtros**, aplicando esses conceitos na construção de um **CRUD funcional e validado**.

PARTE 1 — TEORIA (≈ 2 horas)

1 Pipeline HTTP no ASP.NET Core

O **pipeline HTTP** é uma sequência de **middlewares** que processam a requisição.

Fluxo simplificado:

Request → Middleware → Routing → Controller → View → Response

Cada middleware pode:

- Processar a requisição
 - Encerrar o pipeline
 - Passar para o próximo
-

2 Middleware

Exemplos comuns:

- `UseExceptionHandler`
- `UseStaticFiles`

- `UseRouting`
- `UseAuthentication`
- `UseAuthorization`

Exemplo no `Program.cs` :

```
app.UseRouting(); // Define o sistema de rotas app.UseAuthorization(); // Aplica regras de autorização
```

👉 Ordem importa.

3 Controllers e Actions

- Controller recebe requisições HTTP
- Action executa lógica e retorna uma resposta

Exemplo:

```
public IActionResult Index() { return View(); }
```

Tipos de retorno:

- `View()`
 - `RedirectToAction()`
 - `Json()`
 - `NotFound()`
-

4 Model Binding

O **Model Binding** converte dados da requisição para objetos C#.

Fontes:

- `Route`

- QueryString
- Form
- Body

Exemplo:

```
[HttpPost] public IActionResult Create(Produto produto)
```

5 Validation (DataAnnotations)

Valida automaticamente modelos.

Exemplo:

```
[Required] [StringLength(100)] public string Nome { get; set; }
```

Integração com Razor:

```
<span asp-validation-for="Nome"></span>
```

6 Filtros (Filters)

Executam código antes/depois das Actions.

Tipos:

- Authorization
- Action
- Result
- Exception

Exemplo:

```
public class LogActionFilter : IActionFilter
```

PARTE 2 — PRÁTICA (≈ 2 horas)

Objetivo Prático

Criar um CRUD completo de Produto usando MVC, validações automáticas e Views fortemente tipadas.

1 Criando o Model com Validações

 Models/Produto.cs

```
using System.ComponentModel.DataAnnotations; namespace Aula01.Web.Models { public class Produto { public int Id { get; set; }  
[Required(ErrorMessage = "O nome é obrigatório")] [StringLength(100, MinimumLength = 3)] public string Nome { get; set; }  
[Required] [Range(0.01, 99999)] public decimal Preco { get; set; } public bool Ativo { get; set; } } }
```

2 Criando o Controller CRUD

 Controllers/ProdutoController.cs

```
using Aula01.Web.Models; using Microsoft.AspNetCore.Mvc; namespace Aula01.Web.Controllers { public class ProdutoController :  
Controller { // Simulação de base de dados em memória private static List<Produto> _produtos = new(); private static int _id = 1;  
// LISTAR public IActionResult Index() { return View(_produtos); } // FORMULÁRIO CRIAÇÃO public IActionResult Create() { return  
View(); } // SALVAR [HttpPost] public IActionResult Create(Produto produto) { if (!ModelState.IsValid) return View(produto);  
produto.Id = _id++; _produtos.Add(produto); return RedirectToAction(nameof(Index)); } // FORMULÁRIO EDIÇÃO public IActionResult  
Edit(int id) { var produto = _produtos.FirstOrDefault(p => p.Id == id); if (produto == null) return NotFound(); return
```

```
View(produto); } // ATUALIZAR [HttpPost] public IActionResult Edit(Produto produto) { if (!ModelState.IsValid) return
View(produto); var existente = _produtos.First(p => p.Id == produto.Id); existente.Nome = produto.Nome; existente.Preco =
produto.Preco; existente.Ativo = produto.Ativo; return RedirectToAction(nameof(Index)); } // EXCLUIR public IActionResult
Delete(int id) { var produto = _produtos.FirstOrDefault(p => p.Id == id); if (produto != null) _produtos.Remove(produto); return
RedirectToAction(nameof(Index)); } }
```

3 Views Fortemente Tipadas

Views/Produto/Index.cshtml

```
@model List<Aula01.Web.Models.Produto> <h2>Produtos</h2> <a asp-action="Create">Novo Produto</a> <table> <tr> <th>Nome</th>
<th>Preço</th> <th>Ações</th> </tr> @foreach (var p in Model) { <tr> <td>@p.Nome</td> <td>@p.Preco.ToString("C")</td> <td> <a
asp-action="Edit" asp-route-id="@p.Id">Editar</a> | <a asp-action="Delete" asp-route-id="@p.Id">Excluir</a> </td> </tr> }
</table>
```

Views/Produto/Create.cshtml

```
@model Aula01.Web.Models.Produto <h2>Novo Produto</h2> <form asp-action="Create" method="post"> <label>Nome</label> <input asp-
for="Nome" /> <span asp-validation-for="Nome"></span> <label>Preço</label> <input asp-for="Preco" /> <span asp-validation-
for="Preco"></span> <label> <input asp-for="Ativo" /> Ativo </label> <button type="submit">Salvar</button> </form>
```

Views/Produto/Edit.cshtml

```
@model Aula01.Web.Models.Produto <h2>Editar Produto</h2> <form asp-action="Edit" method="post"> <input type="hidden" asp-for="Id" /> <label>Nome</label> <input asp-for="Nome" /> <span asp-validation-for="Nome"></span> <label>Preço</label> <input asp-for="Preco" /> <span asp-validation-for="Preco"></span> <label> <input asp-for="Ativo" /> Ativo </label> <button type="submit">Salvar</button> </form>
```

4 Ativando Validação Client-Side

_Layout.cshtml

```
<script src="~/lib/jquery/jquery.min.js"></script> <script src="~/lib/jquery-validation/jquery.validate.min.js"></script> <script src="~/lib/jquery-validation-unobtrusive/jquery.validate.unobtrusive.min.js"></script>
```

5 Exemplo de Filtro de Log

Filters/LogActionFilter.cs

```
using Microsoft.AspNetCore.Mvc.Filters; namespace Aula01.Web.Filters { public class LogActionFilter : IActionFilter { public void OnActionExecuting(ActionExecutingContext context) { Console.WriteLine("Executando Action..."); } public void OnActionExecuted(ActionExecutedContext context) { Console.WriteLine("Action finalizada."); } } }
```

Registrar:

```
builder.Services.AddControllersWithViews(options => { options.Filters.Add<LogActionFilter>(); });
```

✓ Resultado Esperado

- Entendimento do pipeline MVC

- CRUD funcional
 - Validação automática
 - Views fortemente tipadas
 - Base sólida para EF Core (Aula 4)
-

Atividade Avaliativa

Desafio:

1. Adicionar campo `Categoria`
2. Validar mínimo de caracteres
3. Criar filtro para logar tempo de execução da Action